OCTOBER 23, 2026

WHEN A VALID SIGNATURE LIES: BLOCKCHAIN QUORUM VERIFICATION AGAINST SIGNED MALWARE IN ANDROID APPS

JATIN KUMAR SINGH
The NorthCap University
Gurugram, Haryana, India

When a Valid Signature Lies: Blockchain Quorum Verification Against Signed Malware in Android Apps

Abstract. Android installs an app only if the app carries a valid signature, and most users and app stores take a valid signature to mean that the app is safe. This is true only when the developer's build system and signing key are safe. Attacks such as SolarWinds, CCleaner and ASUS ShadowHammer show that attackers can break into a build system or steal a signing key and then ship malware that is signed correctly. In such cases the signature check passes and the user has no protection. In this paper we present a lightweight blockchain-based design that removes this single point of trust for Android apps. In our design, a few independent builders build the same app from the same source code and record a signing-independent digest of the APK in a smart contract. A device installs an APK only if at least k of the n builders recorded the same digest, the signing certificate matches, the release is not revoked, and the version is not below a minimum version set by the developer. We built a prototype smart contract and a digest tool and tested them on a local Ethereum network. In all 11 test scenarios, which include build server compromise, a stolen signing key, a repackaged app, a revoked release and a downgrade attempt, the system gave the expected result. One builder vote costs between 55,162 and 72,262 gas, and checking a release is a free read-only call. Computing the digest of a 100 MB test file took about 0.4 seconds on a small cloud machine. We also explain the limits of the design, such as its need for reproducible builds.

Keywords: Blockchain · Smart contracts · Android security · Software supply chain · APK signing · Reproducible builds

1 Introduction

Every Android app is shipped as an APK file, and every APK must be signed by its developer. Before it installs an app, Android checks this signature. If the signature is valid, the app is installed. Because of this, a valid signature has become the main sign of trust for Android apps. App stores, users and many security tools treat a correctly signed app as a genuine app.

However, a signature proves only one thing: the holder of a certain private key signed the file. It does not prove that the code inside the file is the code that the developer wrote. If an attacker controls the build machine or steals the signing key, the attacker can produce malware that carries a perfectly valid signature. This is not only a theory. In 2020, attackers placed the SUNBURST backdoor inside updates of the SolarWinds Orion software, and the backdoored component was digitally signed [1]. The company reported that close to 18,000 customers installed the bad update [2]. In 2017, a version of CCleaner with hidden malware was signed with the vendor's valid certificate and reached about 2.27 million users [3]. In Operation ShadowHammer, attackers pushed a backdoored ASUS Live Update tool from official ASUS servers, signed with real ASUS certificates [4]. In each of these cases, the signature check said that the software was fine.

Android has its own form of this problem. Repackaging is very common: an attacker takes a real app, adds harmful code to it and publishes it again. Zhou and Jiang found that 1,083 of the 1,260 malware samples in their dataset, or 86.0%, were repackaged versions of real apps [5]. A repackaged app is usually signed with the attacker's own key, so any user who downloads it from outside the official store can install it as a new app. The APK signing schemes also give a developer no built-in way to say that one exact release is bad and must not be installed. Finally, the Android package manager refuses to replace an installed app with an older version, but this check works only when a newer version is already on the device. A user who installs an old and vulnerable version for the first time is not stopped.

In this paper we ask a simple question: can we make a valid signature not enough on its own? Our answer is a small blockchain-based system. The main idea is to remove the single point of trust. Instead of trusting one build machine and one key, we ask several independent builders to build the same app from the same source code. Each builder records a digest of the APK in a smart contract. We compute this digest in a way that skips the signature block, so honest builders get the same value even though they do not hold the developer's key. A device accepts an APK only if enough builders agree on its digest. The same contract also stores the developer's certificate, a list of revoked releases and a minimum allowed version.

This paper makes the following contributions:

- We describe a threat model for valid-signature attacks on Android app distribution. It covers build system compromise, signing key theft, repackaging, revoked releases and downgrades.
- We design a smart contract that combines k-of-n builder agreement, a signing-independent APK digest, signer binding, release revocation and a version floor.
- We build a working prototype and test it on a local Ethereum network with 11 attack and access control scenarios. We report measured gas costs and digest computation times.
- We give a security analysis and a clear list of limits, including the attacks that the design does not stop.

The rest of this paper is organised as follows. Section 2 gives background. Section 3 defines the threat model. Section 4 presents the design. Section 5 describes the prototype and the results. Section 6 analyses security. Section 7 covers related work. Section 8 discusses limits and future work, and Section 9 concludes the paper.

2 Background

2.1 APK Signing

An APK is a ZIP file. Android supports several signing schemes. The oldest scheme, v1 or JAR signing, stores signature files inside the ZIP in the META-INF folder. APK Signature Scheme v2, added in Android 7.0, signs the whole file [6]. It adds a new structure called the APK Signing Block just before the ZIP Central Directory. For integrity, v2 treats the APK as four parts: (1) the contents of the ZIP entries, (2) the APK Signing Block, (3) the ZIP Central Directory and (4) the ZIP End of Central Directory (EOCD). The signature protects parts 1, 3 and 4, while part 2 holds the signatures themselves [6]. Scheme v3, added in Android 9, keeps the same layout and adds support for changing the signing key [7].

Two facts matter for our work. First, since the signatures live in part 2, the other three parts do not depend on who signed the file. Second, the platform only checks that the signature is valid. It does not check where the code came from or how it was built.

2.2 Reproducible Builds

A build is reproducible when the same source code always gives the same output, bit for bit. Lamb and Zacchiroli explain that with reproducible builds, users can compare the outputs of several independent builders and trust a binary only when they match [8]. In the Android world, F-Droid already checks reproducible builds by rebuilding apps and comparing the result with the developer's APK [8,9]. F-Droid also documents practical problems, such as ZIP entries stored in a different order and native code that changes across operating systems [9]. Our design depends on reproducible builds, so these problems affect us too, as we discuss in Section 8.

2.3 Blockchain and Smart Contracts

A blockchain is a shared ledger. Once data is written into a confirmed block, it is very hard to change. A smart contract is a program stored on the blockchain that runs in the same way for everyone [10]. On Ethereum, every change of contract state costs gas, while read-only calls are free for the caller. We use a smart contract for two jobs: as a public record that no single party controls, and as a referee that applies the same trust rules for every device.

3 Threat Model

3.1 Actors

Developer. The developer owns the source code and the APK signing key. The developer registers the app in the contract, chooses the builders and the threshold k, and can revoke releases and raise the minimum version.

Builders. There are n independent builders. A builder can be the developer's own CI server, an outside audit company or a community build server. Each builder builds the app from the same tagged source commit and gives one vote for each version.

Distribution channel. This is any app store, website or mirror that serves the APK. We treat it as untrusted.

Verifier. The verifier runs on the user's device before an app is installed.

Blockchain. The blockchain stores the records and runs the contract.

3.2 Attacker Goals and Capabilities

The attacker wants a user's device to install an APK that is different from the honest build of the source code, or to install a release that the developer no longer trusts. The attacker may fully control the distribution channel and the network. We consider five attacks:

- **A1, build server compromise.** The attacker controls one build machine, as in SolarWinds and CCleaner. The output is signed with the real key.
- **A2, signing key theft.** The attacker holds the developer's private key, as in ShadowHammer, and can sign any APK.
- **A3, repackaging.** The attacker changes a real APK, injects code, signs it with a different key and spreads it.
- **A4, revoked release.** The attacker serves a release that the developer has withdrawn, for example because it contains a serious bug.
- **A5, downgrade.** The attacker serves an older version with a known weakness to a user who does not have the app yet.

3.3 Assumptions

Our design is secure under the following assumptions:

- At most k - 1 of the n builders are compromised at the same time, and the builders do not share infrastructure.
- The source code at the tagged commit is correct. We do not detect harmful code that is committed into the source repository itself.
- The app builds reproducibly, so honest builders get the same digest.
- The blockchain works correctly: confirmed records cannot be changed, and the deployed contract is the code we describe.
- The verifier on the device is not compromised, and it reads the chain through a node or service that it trusts.
- The developer's blockchain account key, used to manage the app, is stored separately from the APK signing key.

3.4 Out of Scope

We do not cover bugs or backdoors in the source code, runtime attacks on an installed app such as injection or reading of app data, a fully compromised device, or a group of k or more builders working together against the user.

4 System Design

4.1 Overview

Fig. 1 shows the design. The release process has four steps. (1) The developer registers the app once in the contract with the list of builders, the threshold k and the hash of the signing certificate. (2) For each new version, the developer tags a commit. Each builder builds that commit, computes the digest D of the APK and sends it to the contract. (3) The developer signs the APK and publishes it through any channel. (4) Before installation, the verifier on the device computes D and the certificate hash C from the APK and asks the contract. If any check fails, the installation is blocked and the user sees the reason.

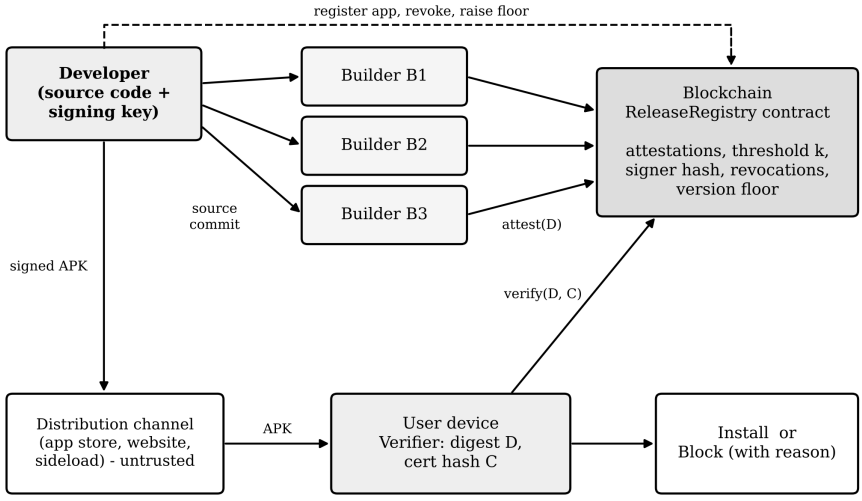


Fig. 1. Overall architecture. Solid arrows show the normal release and install flow. The dashed arrow shows management calls that only the developer can make.

4.2 Signing-Independent Digest

Builders must not hold the developer's signing key, because sharing the key would create the same single point of failure we want to remove. As a result, the file produced by a builder is unsigned or signed with a different key, and its plain file hash is different from the hash of the APK that the developer publishes. To solve this, we use the same split of the APK as scheme v2. Let P1 be the ZIP entry data, P3 the Central Directory and P4 the EOCD record. We define:

$$D = \text{SHA-256}(P1 \,||\, P3 \,||\, P4^*)$$

Here || means joining bytes together, and P4* is the EOCD record in which the Central Directory offset is replaced by the offset where the signing block starts. Scheme v2 uses the same rule for the same reason [6]. As a result, D is the same for an unsigned APK and for the same APK signed by any key, but it changes if even one byte of the ZIP entries, the Central

Directory or the EOCD changes. The certificate hash C is the SHA-256 hash of the signing certificate taken from the v2 or v3 block.

This digest needs apps that are signed only with scheme v2 or later. With v1 signing, the signature files sit inside P1 and differ for each signer. We discuss this limit in Section 8.

4.3 Smart Contract

The contract, called ReleaseRegistry, stores for each app its owner, the threshold k, the number of builders, the minimum version and the certificate hash. It also stores the list of builders, a vote counter for each pair of version and digest, a record of which builder has voted for which version, and the set of revoked versions. The app is identified by the Keccak-256 hash of its package name. Table 1 lists the functions.

Table 1. Functions of the ReleaseRegistry contract.

Function	Caller	Effect
registerApp(pkg, builders, k, C)	Anyone (becomes owner)	Creates the app record
attest(app, version, D)	Registered builder	Adds one vote for D
revoke(app, version)	Owner	Marks the release as revoked
raiseVersionFloor(app, v)	Owner	Sets a higher minimum version
verify(app, version, D, C)	Anyone, read-only	Returns a status code

We made four design choices on purpose. First, each builder can vote only once for each version. A compromised builder therefore cannot vote twice for its own digest, and it cannot vote for many digests at the same time. Second, the version floor can only go up. Even if the owner account is stolen, the attacker cannot lower the floor to allow old versions again. Third, the owner has no voting power unless it is also listed as a builder, so the owner account alone cannot make an APK trusted. Fourth, verify is a read-only call, so checking an app costs the user nothing.

4.4 Verification

Fig. 2 shows the checks that the verifier runs. Given an APK, the verifier reads the package name and the version code, computes D and C, and calls verify. The contract returns OK only if all of the following are true:

1. The app is registered. Otherwise the result is UNKNOWN_APP.
2. C equals the registered certificate hash. Otherwise the result is SIGNER_MISMATCH.
3. The version is not revoked. Otherwise the result is REVOKED.
4. The version is not below the floor. Otherwise the result is BELOW_VERSION_FLOOR.
5. At least k builders voted for D for this version. Otherwise the result is QUORUM_NOT_REACHED.

The verifier also runs the normal Android signature check, so a file with a broken signature is still rejected. The order of the checks only changes which reason the user sees, because all of them must pass.

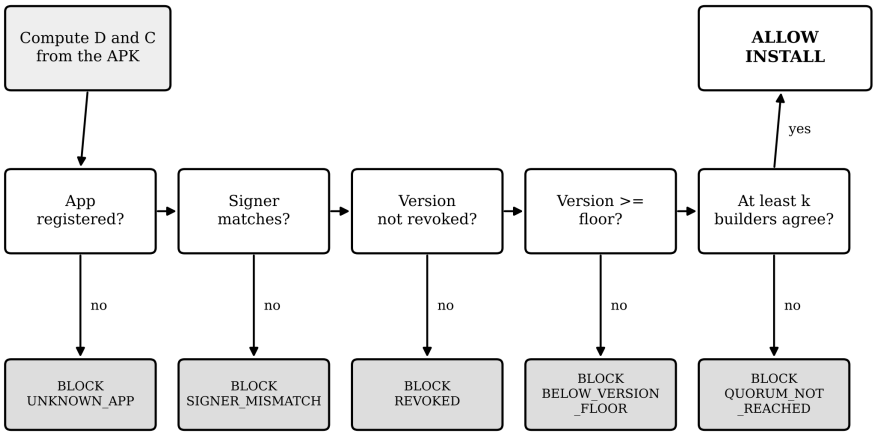


Fig. 2. Checks run before an APK is installed. Each failed check blocks the installation and shows the reason.

5 Implementation and Evaluation

5.1 Prototype and Setup

We wrote the contract in Solidity 0.8.24 with the optimizer turned on (200 runs) and tested it on the built-in local network of Hardhat 2.22.17. The digest tool is a short Python 3.13 program that follows the rule in Section 4.2. All tests ran on a cloud machine with an x86_64 processor and 2 virtual CPUs. We did not use a public network or real money.

Our tests have two limits that the reader should know. In the contract tests, the release digests are fixed test values and not digests of real APKs. In the digest tests, we used synthetic APK-like ZIP files that contain random data and a fake APK Signing Block placed in the same position as in a real APK. Tests with real APKs are future work (Section 8).

5.2 Attack Scenarios

We registered a test app with three builders and k = 2. For version 5, builders B1 and B2 voted for the clean digest, and builder B3, playing the role of a compromised builder, voted for a backdoored digest. Version 4 received three votes and was then revoked. Version 3 received two votes, and the floor was then raised to 4. Table 2 lists the 11 scenarios. All of them gave the expected result.

Table 2. Results of the attack and access control scenarios on the local network.

ID	Scenario	Expected	Result
S1	Clean v5, 2 of 3 builders agree	OK	Pass
S2	A1: backdoored v5 with real certificate, 1 of 3 votes	QUORUM_NOT_REACHED	Pass
S3	A2: stolen key, attacker's own build, no votes	QUORUM_NOT_REACHED	Pass
S4	A3: repackaged app signed with another key	SIGNER_MISMATCH	Pass
S5	A4: revoked release v4	REVOKED	Pass
S6	A5: old v3 below the floor	BELOW_VERSION_FLOOR	Pass
S7	Package that is not registered	UNKNOWN_APP	Pass
S8	Outsider tries to vote	Reverted	Pass
S9	Builder tries to vote twice for v5	Reverted	Pass
S10	Outsider tries to revoke a release	Reverted	Pass
S11	Owner tries to lower the floor	Reverted	Pass

5.3 Gas Cost

Table 3 shows the gas used by each operation, as reported in the transaction receipts. Deployment happens once. Registration happens once per app and grows by about 23,450 gas for each extra builder. The first vote for a new digest costs more than later votes because it creates a new storage slot. With k = 2, a release reached quorum after two votes that cost 127,424 gas in total. The cost does not depend on the size of the APK, because the contract stores only a 32-byte digest. The verify call is free for the user. We do not convert gas into money, because gas prices change all the time and differ a lot between networks.

Table 3. Measured gas cost of contract operations.

Operation	Gas used
Deploy contract (once)	756,689
registerApp with n = 3 builders	163,213
registerApp with n = 5 builders	210,119
registerApp with n = 7 builders	257,025
attest, first vote for a digest	72,262
attest, vote for a digest that already has a vote	55,162
revoke	48,116
raiseVersionFloor	29,119
verify (read-only call)	free (34,508 if used inside a transaction)

5.4 Digest Correctness and Speed

We first checked that the digest behaves as intended on a 2 MB test file. The digest of the unsigned file was equal to the digest of the same file with a signing block for key A. The digest with key A was equal to the digest with a different signing block for key B. When we changed four bytes inside the code entry, the digest changed. We then measured the time to compute the digest for files of 5 MB to 100 MB. Each size was run seven times. Table 4 gives the median. The time grows roughly in line with the file size, and a 100 MB file took about 0.4 seconds. The digest is computed only once per installation. These numbers come from a cloud machine and not from a phone, so on-device timing is part of our future work.

Table 4. Time to compute the digest (median of 7 runs, synthetic files).

File size	Median (ms)	Min (ms)	Max (ms)
5 MB	21.0	14.2	25.8
25 MB	68.3	67.4	74.7
50 MB	187.7	167.8	193.2
100 MB	391.4	334.8	423.7

6 Security Analysis

We now explain why each attack from Section 3 fails under our assumptions. Table 5 gives a summary.

A1, build server compromise. A compromised builder produces a backdoored APK with digest D' that differs from the honest digest D. Its vote counts only for D'. Since at most k - 1 builders are compromised, D' gets at most k - 1 votes, so the result is QUORUM_NOT_REACHED (scenario S2). If the attacker instead breaks into the developer's final signing step, the published APK has a digest with no votes and is also blocked.

A2, signing key theft. With the stolen key, the attacker can produce an APK with the correct certificate hash C. However, no honest builder voted for the attacker's digest, so the APK is blocked (S3). This is the main gain over plain signing: stealing the key is no longer enough, and the attacker must also control k builders.

A3, repackaging. A repackaged APK signed with another key fails the certificate check (S4). If the attacker also had the real key, the injected code would still change D, and the case becomes the same as A2.

A4, revoked release. A revoked release is rejected as soon as the revoke transaction is confirmed (S5). Because the revocation is public, every verifier sees it.

A5, downgrade. An APK whose version is below the floor is rejected (S6). Unlike the built-in Android check, this also works for a first-time installation.

Misuse of roles. Outsiders cannot vote (S8), a builder cannot vote twice (S9), and outsiders cannot revoke releases (S10). If the owner account is stolen, the attacker can revoke releases or raise the floor, which can stop users from installing the app. This is a denial of service, but it cannot make malware pass, because the owner cannot add votes and cannot lower the floor (S11).

Table 5. Summary of attacks against plain APK signing and against our design.

Attack	Plain APK signing	Our design
A1 build server	Installed	Blocked if fewer than k builders are compromised
A2 key theft	Installed	Blocked unless k builders are also compromised
A3 repackaging	Installed as new app	Blocked (signer mismatch)
A4 revoked release	Installed	Blocked
A5 downgrade (new user)	Installed	Blocked

7 Related Work

Decentralised software updates. CHAINIAC is the work closest to ours [11]. It uses independent witness servers that collectively sign releases, build verifiers that check that binaries match the source, and a tamper-proof release log built as a skipchain. It was evaluated on Debian packages and PyPI data. Our design shares the main goal of removing single points of failure. It differs in three ways: it uses a general-purpose smart contract instead of a custom collective signing protocol,

it targets Android APKs with a digest that does not depend on the signer, and it adds an explicit version floor and per-release revocation in the same contract. Our design is smaller and less general than CHAINIAC.

Supply chain verification. in-toto lets a project owner define the steps of a supply chain and checks signed evidence from each step [12]. In its Debian deployment, a package is trusted only if enough independent rebuilders agree, which is the same threshold idea that we use. Sigstore makes signing easy by binding signatures to identities and recording them in public transparency logs [13]. These logs show who signed an artifact, but they do not require several builders to agree, and they are run by a log operator. Our design puts the record and the trust rule on a blockchain instead.

Reproducible builds and transparency for Android. The Reproducible Builds project [8] and F-Droid [9] give the base that our design needs. Google publishes a Pixel Binary Transparency log that records metadata of official Pixel factory images, so users can check that an image is genuine [14]. That log covers operating system images from one publisher, while our design covers apps and asks for agreement between several builders.

Blockchain for APK integrity. Lee and Lee built a DApp on a Hyperledger consortium blockchain in which an administrator registers information about legitimate APKs, and users then check whether an APK is forged [15]. In their system one administrator decides which APKs are legitimate. Our design instead needs several independent builders to agree, and it also handles revocation and downgrades.

8 Limitations and Future Work

Our design has several limits.

- **Reproducible builds.** The design works only when honest builders get bit-for-bit equal APKs. F-Droid reports that Android builds often differ because of ZIP entry order, some generated files and native code built on different systems [9]. If a build is not reproducible, honest builders disagree and the release cannot reach quorum. Shared, pinned build containers can reduce this problem.
- **v1 signing.** Our digest assumes v2 or later signing only. Supporting v1 would need the digest to skip the META-INF signature files.
- **Package name squatting.** In our prototype, the first account that registers a package name becomes its owner. A real system needs proof of ownership, for example a registration message signed with the APK signing key.
- **Key and builder changes.** The prototype cannot change the builder list or the certificate hash. Scheme v3 key rotation [7] is therefore not supported yet.
- **Test coverage.** We tested the contract with fixed test digests and the digest tool with synthetic files on a cloud machine. We have not yet tested real APKs or real phones.
- **Scope.** The design does not stop harmful code that is committed into the source, attacks on a running app, or a group of k colluding builders. Release records are public, which is fine for public apps but may not suit private enterprise apps.

In future work we plan to build the verifier as an Android installer app, test the system with real apps that F-Droid already builds reproducibly, deploy the contract on a public test network to measure delays and costs, and add safe ways to change keys and builders.

9 Conclusion

A valid signature tells us who signed an app, but not whether the app was built honestly. Past attacks show that a single compromised build server or a single stolen key is enough to ship signed malware to millions of users. We presented a lightweight blockchain-based design in which an Android app is trusted only when several independent builders agree on a signing-independent digest of the APK, the signer matches, the release is not revoked and the version is not below a floor. Our prototype blocked all tested attacks, needs only a few contract calls per release and gives users a free check before installation. The design does not replace APK signing. It adds a second, independent check, so that a valid signature alone is no longer enough to install an app.

References

1. FireEye: Highly evasive attacker leverages SolarWinds supply chain to compromise multiple global victims with SUNBURST backdoor. FireEye Threat Research Blog (13 December 2020)
2. SolarWinds Corporation: Form 8-K, current report. U.S. Securities and Exchange Commission (14 December 2020)
3. Cisco Talos: CCleanup: A vast number of machines at risk. Cisco Talos Intelligence Blog (18 September 2017)
4. Kaspersky: Operation ShadowHammer. Securelist (25 March 2019), https://securelist.com/operation-shadowhammer/89992/
5. Zhou, Y., Jiang, X.: Dissecting Android malware: characterization and evolution. In: 2012 IEEE Symposium on Security and Privacy, pp. 95-109. IEEE (2012). https://doi.org/10.1109/SP.2012.16

6. Android Open Source Project: APK signature scheme v2, https://source.android.com/docs/security/apksigning/v2, last accessed 2026/10/22
7. Android Open Source Project: APK signature scheme v3, https://source.android.com/docs/security/apksigning/v3, last accessed 2026/10/22
8. Lamb, C., Zacchiroli, S.: Reproducible builds: increasing the integrity of software supply chains. IEEE Software 39(2), 62-70 (2022). https://doi.org/10.1109/MS.2021.3073045
9. F-Droid: Reproducible builds, https://f-droid.org/docs/Reproducible_Builds/, last accessed 2026/10/22
10. Buterin, V.: Ethereum: a next-generation smart contract and decentralized application platform. White paper (2014)
11. Nikitin, K., Kokoris-Kogias, E., Jovanovic, P., Gailly, N., Gasser, L., Khoffi, I., Cappos, J., Ford, B.: CHAINIAC: proactive software-update transparency via collectively signed skipchains and verified builds. In: 26th USENIX Security Symposium, pp. 1271-1287. USENIX Association (2017)
12. Torres-Arias, S., Afzali, H., Kuppusamy, T.K., Curtmola, R., Cappos, J.: in-toto: providing farm-to-table guarantees for bits and bytes. In: 28th USENIX Security Symposium, pp. 1393-1410. USENIX Association (2019)
13. Newman, Z., Meyers, J.S., Torres-Arias, S.: Sigstore: software signing for everybody. In: 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS). ACM (2022)
14. Google: Pixel binary transparency, https://developers.google.com/android/binary_transparency/pixel_overview, last accessed 2026/10/22
15. Lee, H.-W., Lee, H.: Consortium blockchain based forgery Android APK discrimination DApp using Hyperledger Composer. Journal of Internet Computing and Services 20(5), 9-18 (2019). https://doi.org/10.7472/jksii.2019.20.5.9